

BackendJS API

Guía Completa de Uso con Thunder Client

0. Requisitos Previos

Antes de comenzar, asegúrate de tener lo siguiente:

- Node.js v18+ instalado (o Docker Desktop corriendo)
- Extensión Thunder Client instalada en VS Code
- El servidor de la API corriendo en puerto 3000
- Acceso a la base de datos MySQL (Railway o local)

1. Levantar el Servidor

1.1 Con Docker (Recomendado)

Abre una terminal en la carpeta raíz del proyecto y ejecuta los siguientes comandos en orden:

Paso 1 — Limpiar contenedores anteriores

```
docker-compose down
docker system prune -f
```

Paso 2 — Construir la imagen

```
docker build -t mi-api-node .
```

Este comando puede tardar varios minutos la primera vez (descarga la imagen de Node.js 18).

Paso 3 — Levantar el contenedor en segundo plano

```
docker-compose up -d
```

Paso 4 — Verificar que está corriendo

```
docker-compose logs -f app
```

Deberías ver:  Conectado a MySQL y  Servidor en http://localhost:3000

1.2 Sin Docker (Node.js directo)

```
npm install
node index.js
```

1.3 Verificar que la API responde

Abre Thunder Client y crea una nueva solicitud:

Campo	Valor
Método	GET
URL	http://localhost:3000/
Headers	(ninguno)
Body	(vacío)

Respuesta esperada:

```
{ "message": "🚀 API funcionando correctamente" }
```

2. Registrar un Usuario Nuevo

Antes de iniciar sesión necesitas una cuenta. Si ya tienes una, pasa a la Sección 3.

Campo	Valor
Método	POST
URL	http://localhost:3000/auth/register
Header	Content-Type: application/json
Body (JSON)	Ver ejemplo abajo

Body JSON a enviar:

```
{
  "email": "tu_correo@ejemplo.com",
  "password": "tu_contraseña123"
}
```

Pasos en Thunder Client:

- Abre Thunder Client → New Request
- Cambia el método a POST
- Escribe la URL: http://localhost:3000/auth/register

- Ve a la pestaña Headers → agrega: Content-Type = application/json
- Ve a la pestaña Body → selecciona JSON → pega el body de arriba
- Haz clic en Send

Respuesta exitosa (201):

```
{ "message": "Usuario creado exitosamente" }
```

Posibles errores:

Código	Mensaje	Causa
400	Email y contraseña requeridos	Falta algún campo en el body
409	El email ya está registrado	Ese correo ya existe en la base de datos
500	Error interno del servidor	Problema con la base de datos

3. Iniciar Sesión (Obtener Token JWT)

IMPORTANTE: Este paso es OBLIGATORIO antes de usar cualquier operación con ítems. El token que obtienes aquí lo usarás en todas las demás solicitudes.

Campo	Valor
Método	POST
URL	http://localhost:3000/auth/login
Header	Content-Type: application/json
Body (JSON)	Ver ejemplo abajo

Body JSON a enviar:

```
{
  "email": "tu_correo@ejemplo.com",
  "password": "tu_contraseña123"
}
```

Pasos detallados en Thunder Client:

- Abre Thunder Client → New Request
- Cambia el método a POST
- Escribe la URL: http://localhost:3000/auth/login
- Ve a Headers → agrega: Content-Type = application/json
- Ve a Body → selecciona JSON → pega el body
- Haz clic en Send

- COPIA el token de la respuesta (lo necesitarás en los siguientes pasos)

Respuesta exitosa (200):

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MS...",
  "message": "Login exitoso"
}
```

El token tiene vigencia de 24 horas. Después de ese tiempo deberás hacer login nuevamente.

Posibles errores:

Código	Mensaje	Causa
400	Email y contraseña requeridos	Falta algún campo
401	Usuario no encontrado	El email no existe
401	Contraseña incorrecta	La contraseña no coincide

4. Cómo Usar el Token en Thunder Client

Para TODAS las operaciones con ítems (Secciones 5–8) debes incluir el token en el header de cada solicitud. Sigue estos pasos:

- En Thunder Client, ve a la pestaña Headers de tu solicitud
- Agrega un nuevo header con estos valores exactos:

Header Key	Header Value
Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... (tu token completo)

Importante: el prefijo Bearer (con espacio) es obligatorio antes del token. También puedes pegar solo el token sin el prefijo Bearer, ambos formatos son aceptados.

5. Obtener Todos los Ítems

Campo	Valor
Método	GET

URL	http://localhost:3000/api/items
Header requerido	Authorization: Bearer <tu_token>
Body	(vacío — GET no lleva body)

Pasos en Thunder Client:

- New Request → método GET
- URL: http://localhost:3000/api/items
- Headers → Authorization = Bearer <pega_tu_token_aquí>
- Send

Respuesta exitosa (200) — arreglo de ítems:

```
[
  {
    "id": 1,
    "nombre": "Laptop Dell",
    "descripcion": "Laptop para desarrollo",
    "estado": 1,
    "created_at": "2026-04-20T15:30:00.000Z"
  },
  ...
]
```

6. Obtener un Ítem por ID

Campo	Valor
Método	GET
URL	http://localhost:3000/api/items/1 (reemplaza 1 con el ID)
Header requerido	Authorization: Bearer <tu_token>
Body	(vacío)

Pasos en Thunder Client:

- New Request → método GET
- URL: http://localhost:3000/api/items/1 (cambia el 1 por el ID que quieres buscar)
- Headers → Authorization = Bearer <tu_token>
- Send

Respuesta exitosa (200):

```
{
  "id": 1,
  "nombre": "Laptop Dell",
  "descripcion": "Laptop para desarrollo",
  "estado": 1,
  "created_at": "2026-04-20T15:30:00.000Z"
}
```

Si el ID no existe: responde 404 con { "message": "Item no encontrado" }

+ 7. Crear un Nuevo Ítem

Campo	Valor
Método	POST
URL	http://localhost:3000/api/items
Header 1	Content-Type: application/json
Header 2	Authorization: Bearer <tu_token>
Body (JSON)	Ver ejemplo abajo

Body JSON a enviar:

```
{
  "nombre": "Monitor Samsung",
  "descripcion": "Monitor 27 pulgadas 4K",
  "estado": 1
}
```

Campos del body:

Campo	Requerido	Tipo	Descripción
nombre	SÍ	string	Nombre del ítem (máximo 100 caracteres)
descripcion	NO	string	Descripción detallada del ítem
estado	NO	boolean (0/1)	1 = activo, 0 = inactivo. Por defecto: 1

Pasos en Thunder Client:

- New Request → método POST
- URL: http://localhost:3000/api/items
- Headers → agrega Content-Type = application/json
- Headers → agrega Authorization = Bearer <tu_token>

- Body → selecciona JSON → pega el body
- Send

Respuesta exitosa (201):

```
{
  "message": "Item creado",
  "id": 5
}
```

El campo id retornado es el ID del nuevo ítem en la base de datos. Guárdalo para editar o eliminar.

8. Editar un Ítem Existente

Campo	Valor
Método	PUT
URL	http://localhost:3000/api/items/1 (reemplaza 1 con el ID)
Header 1	Content-Type: application/json
Header 2	Authorization: Bearer <tu_token>
Body (JSON)	Ver ejemplo abajo

Body JSON a enviar (todos los campos son requeridos en el PUT):

```
{
  "nombre": "Monitor Samsung Actualizado",
  "descripcion": "Monitor 32 pulgadas 4K HDR",
  "estado": 1
}
```

Pasos en Thunder Client:

- New Request → método PUT
- URL: http://localhost:3000/api/items/1 (cambia el número por el ID del ítem a editar)
- Headers → Content-Type = application/json
- Headers → Authorization = Bearer <tu_token>
- Body → JSON → pega el body con los nuevos valores
- Send

Respuesta exitosa (200):

```
{ "message": "Item actualizado exitosamente" }
```

Posibles errores:

Código	Mensaje	Causa
400	El campo nombre es requerido	No enviaste el campo nombre en el body
404	Item no encontrado	El ID no existe en la base de datos
403	Token inválido o expirado	Tu token venció o está mal escrito

✕ 9. Eliminar un Ítem

Campo	Valor
Método	DELETE
URL	http://localhost:3000/api/items/1 (reemplaza 1 con el ID)
Header requerido	Authorization: Bearer <tu_token>
Body	(vacío — DELETE no lleva body)

Pasos en Thunder Client:

- New Request → método DELETE
- URL: http://localhost:3000/api/items/1 (cambia el número por el ID del ítem a eliminar)
- Headers → Authorization = Bearer <tu_token>
- Send

Respuesta exitosa (200):

```
{ "message": "Item eliminado exitosamente" }
```

ADVERTENCIA: Esta acción es irreversible. El ítem se borra permanentemente de la base de datos.

10. Resumen de Todos los Endpoints

Método	URL	Auth	Descripción
GET	http://localhost:3000/	No	Verificar que la API está activa
POST	http://localhost:3000/auth/register	No	Registrar usuario nuevo

POST	http://localhost:3000/auth/login	No	Iniciar sesión y obtener token
GET	http://localhost:3000/api/items	Sí	Obtener todos los ítems
GET	http://localhost:3000/api/items/:id	Sí	Obtener un ítem por ID
POST	http://localhost:3000/api/items	Sí	Crear un nuevo ítem
PUT	http://localhost:3000/api/items/:id	Sí	Editar un ítem existente
DELETE	http://localhost:3000/api/items/:id	Sí	Eliminar un ítem

11. Comandos Docker de Referencia

Ciclo normal de trabajo

Comando	Qué hace
docker-compose up -d	Levanta el contenedor en segundo plano
docker-compose logs -f app	Ver logs en tiempo real (Ctrl+C para salir)
docker-compose down	Detener y eliminar el contenedor
docker ps	Ver contenedores corriendo actualmente
docker logs node_app	Ver logs del contenedor
docker exec -it node_app bash	Entrar al contenedor (terminal bash)

Reconstruir desde cero (cuando cambias el código)

```
docker-compose down
docker system prune -f
docker build -t mi-api-node .
docker-compose up -d
```

Si aparece el error read-only file system

```
# Opción 1: Reiniciar Docker Desktop completamente

# Opción 2: Limpiar datos corruptos (PowerShell como Admin)
docker system prune -a --volumes -f
docker build -t mi-api-node .
```

12. Errores Comunes y Soluciones

Error	Causa	Solución
401 - Token requerido	No incluiste el header Authorization	Agrega el header Authorization: Bearer <token>
403 - Token inválido	El token venció o está mal copiado	Haz login nuevamente para obtener un token fresco
404 - Ruta no encontrada	La URL está mal escrita	Verifica la URL: /api/items (no /items)
400 - nombre requerido	No enviaste el campo nombre en el body	Incluye nombre en el JSON del body
409 - Email ya registrado	El correo ya existe en la BD	Usa otro correo o haz login directamente
500 - Error interno	Problema con la base de datos	Verifica que el contenedor esté corriendo con docker ps
ECONNREFUSED	El servidor no está corriendo	Ejecuta docker-compose up -d primero